

Exercice 6 — Comprendre **box-sizing**

 **DURÉE** : ● 5 min | ● 10 min | ● 20 min
 **Niveau Bloom** : Appliquer (3) / Analyser (4)
 **Prérequis** : Leçon VI (Introduction CSS, box-model)

Objectif

Distinguer **content-box** et **border-box** et **prédire** la taille réelle d'un élément selon ses propriétés.

● Échauffement

Recopier ce HTML+CSS et observer le rendu :

```
<div class="boite type-a">Boîte A (content-box)</div>  
<div class="boite type-b">Boîte B (border-box)</div>
```

```
/* Gabarit commun – tout ce qui ne change pas d'une boîte à l'autre */  
.boite {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
  margin-bottom: 20px;  
}  
  
/* Seul box-sizing diffère vraiment ; les couleurs servent à distinguer A et B visuellement */  
.type-a {  
  box-sizing: content-box;  
  border-color: red;  
  background: lightyellow;  
}  
  
.type-b {  
  box-sizing: border-box;  
  border-color: blue;  
  background: lightcyan;  
}
```

Ouvrir dans le navigateur → les 2 boîtes ont été déclarées `width: 200px` mais diffèrent :

- **Boîte A** : 250 px de large (200 + 2×20 padding + 2×5 border).
- **Boîte B** : 200 px de large pile (contenu réduit automatiquement).

👉 C'est exactement la différence `content-box` vs `border-box` .

💡 **BONNE PRATIQUE** : `.boite` porte le gabarit commun (taille, padding, marges), `.type-a` / `.type-b` ne portent **que ce qui varie**. Au lieu de dupliquer 5 propriétés sur chaque classe, on factorise — principe DRY (*Don't Repeat Yourself*).

● Normal — À vous de jouer

Pour chaque cas, **calculer la largeur totale occupée** (y compris marges) **sans ouvrir le navigateur** :

Cas A — content-box (défaut)

```
.box-a {  
  box-sizing: content-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
  margin: 10px;  
}
```

Largeur totale ? _____ px

Cas B — border-box

```
.box-b {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
  margin: 10px;  
}
```

Largeur totale ? _____ px, Largeur du contenu interne ? _____ px

Cas C — border-box global

```
*, *::before, *::after { box-sizing: border-box; }  
  
.box-c {  
  width: 300px;  
  padding: 0 40px;  
  border: 2px solid;  
  margin: 0 20px;  
}
```

Largeur totale ? _____ px, Contenu interne ? _____ px

Cas D — piège content-box + %

```
.box-d {  
  box-sizing: content-box;  
  width: 100%;  
  padding: 20px;  
  border: 5px solid;  
}
```

Placé dans un parent de 400 px. **Y a-t-il un problème ? Si oui, lequel ?**

● Défi — Refactoring et test sur projet réel

Reprendre un projet existant (ex : votre page produit du J1) et :

1. Ajouter en tête du CSS :

```
*, *::before, *::after { box-sizing: border-box; }
```

2. Identifier **3 éléments** dont le layout a changé après l'ajout du reset. Documenter ce qui a changé visuellement et pourquoi.
3. Créer une grille 3 colonnes avec `width: 33.33%` + `padding: 1rem` sur chaque colonne. Observer le résultat **avec et sans** `box-sizing: border-box`.